
AI BlogNest API

software project documention

Project Documentation

Project Name	AI BlogNest API
Team ID	SWTID-2026-1876
Date	29 September 2026

1. Project Description

AI BlogNest API is a backend application that provides intelligent blog creation and management with the help of Google Gemini AI. The application allows users to register, log in, create and manage blogs, and also generate AI-powered blog content and summaries. It uses Node.js, Express.js, MongoDB and Mongoose for backend development and follows a modular structure for better maintainability and scalability.

2. Objectives of the Project

- To develop a structured backend API for a blog application.
- To provide secure user registration and login functionality.
- To implement authentication using JWT.
- To securely store user passwords using bcrypt.
- To provide blog creation and management functionality.
- To retrieve and manage blog data using MongoDB.
- To integrate Google Gemini AI with the backend application.
- To generate blog content automatically using AI.
- To summarize existing blog content using AI.
- To develop a modular and maintainable backend system.
- To provide APIs that can be easily tested using API testing tools.

3. Technologies Used

Technology	Purpose
Node.js	Runtime environment for executing the backend application.
Express.js	Server framework to create RESTful APIs.
MongoDB	Database for storing user and blog information.
Mongoose	Connects Node.js with MongoDB and manages database models.
Google Gemini AI	Provides AI-powered blog generation and summarization.
JWT	User authentication and protecting secured API operations.
bcrypt	Securely hashes user passwords.
dotenv	Manages environment variables.
Thunder Client	Used for testing and verifying REST APIs.
Visual Studio Code	Development environment.
GitHub	Source-code management.

4. System Architecture

AI BlogNest API follows a modular backend architecture. The client sends requests through REST APIs to the Express.js server. Requests are processed through routes and middleware and then handled by the appropriate controllers and services. Database operations use Mongoose and MongoDB, while AI-related operations are handled through the services layer connected to Google Gemini AI.

System Flow: Client → Express.js Server → Routes → Middleware → Controllers/Services → MongoDB or Gemini AI → Response

5. File / Folder Structure

```
AI BlogNest API
├── src
│   ├── controllers
│   │   └── blogController.js
│   ├── middleware
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models
│   │   ├── Blog.js
│   │   └── User.js
│   ├── routes
│   │   ├── aiRoutes.js
│   │   ├── authRoutes.js
│   │   └── blogRoutes.js
│   ├── services
│   ├── app.js
│   └── server.js
├── .env
├── .gitignore
├── package.json
├── package-lock.json
├── README.md
├── Thunder Client API Collection
└── Thunder Client Environment
```

6. Features

- User registration and login.
- Password protection using bcrypt.
- JWT-based authentication and protected API access.
- Create, view, update, and delete blog posts.
- AI-powered blog generation using Google Gemini AI.
- AI-powered blog summarization.
- MongoDB database integration with Mongoose models.
- Centralized error-handling middleware.
- REST API testing using Thunder Client.
- JSON-based request and response handling.

7. Project Setup

- Install Node.js and npm.
- Open the AI BlogNest API project in Visual Studio Code.
- Install the required project dependencies.
- Configure the .env file with server port, MongoDB connection string, JWT secret, and Gemini AI configuration.
- Configure the MongoDB connection string.
- Start the backend server using the configured npm development command.
- Test the APIs using Thunder Client.

Milestone 1: Project Setup and Architecture

The first milestone establishes the backend structure, required technologies, environment configuration, database connection, and the overall API request flow.

Activity 1.1: Define the Backend Architecture

- Identify authentication, blog management, AI services, database models, routes, and middleware.
- Separate responsibilities across routes, controllers, services, models, and middleware.
- Define the REST API request and response flow.
- Plan communication between the backend, MongoDB, and Gemini AI.

Activity 1.2: Configure the Development Environment

- Install Node.js and npm.
- Open the project in Visual Studio Code.
- Install project dependencies using npm.
- Create and configure the .env file.

Activity 1.3: Configure MongoDB

- Configure the MongoDB connection string.
- Connect the Node.js application to MongoDB using Mongoose.
- Prepare User and Blog models.

Milestone 2: Authentication Development

This milestone implements secure user registration and login using bcrypt and JWT.

User Registration

- Receive and validate registration details.
- Hash the password using bcrypt.
- Store user information in MongoDB.

User Login

- Receive login credentials.
- Verify the registered user's credentials.
- Generate a JWT token after successful authentication.

Milestone 3: Blog Management Development

The Blogs API provides the main blog-management operations and communicates with MongoDB through Mongoose.

Create Blog

- Accept blog information from an authenticated request.
- Validate the required blog data.
- Store the blog document in MongoDB.

View / Fetch Blog

- Retrieve stored blog information from MongoDB.
- Return the blog content through a JSON API response.

Update Blog

- Identify the required blog record.
- Update the permitted blog information.
- Return the updated result through the API.

Delete Blog

- Identify the selected blog record.
- Remove the record from MongoDB.
- Return an appropriate API response.

Milestone 4: AI Integration

Google Gemini AI is integrated to provide automatic blog generation and blog summarization.

Generate Blog API

- Receive topic and category information.
- Process the input through the AI service layer.
- Send the structured request to Google Gemini AI.
- Return the generated blog content.

Summarize Blog API

- Receive existing blog content.
- Send the content to Gemini AI.
- Generate a short and meaningful summary.
- Return the summary through the API response.

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not specified in the supplied project documentation.
Login Credentials (Demo)	Not specified in the supplied project documentation.
Platform / Hosting Provider	Backend execution and testing are documented locally; cloud deployment is listed as a future enhancement.
Repository Link	A specific repository URL is not specified in the supplied documentation.
Demo Video Link	https://drive.google.com/file/d/1MhCqIg10Q5YqpqMNdZpT8p_pwEBKDYQM/view?usp=drive

API Development

The application follows a RESTful API structure with separate authentication, blog, and AI-related routes.

Route / Operation	Purpose
Register API	Creates a new user account and stores user information.
Login API	Authenticates a registered user and generates a JWT token.
Blogs API	Provides create, view, update, and delete operations.
Generate Blog API	Generates blog content using Google Gemini AI.
Summarize Blog API	Creates a concise summary from existing blog content.
Fetch Blog API	Retrieves stored blog information from MongoDB.

Middleware Development

Middleware functions process requests before they reach the main application logic.

Authentication Middleware

authMiddleware.js verifies the JWT token provided by the user and allows authorized requests to continue.

Error Middleware

errorMiddleware.js provides centralized handling of application errors and appropriate API responses.

Middleware Flow: Client Request → Authentication Middleware → Route → Controller/Service → Response

Database Development

MongoDB is used as the database for AI BlogNest API. Mongoose establishes communication between the Node.js application and MongoDB.

Model	Purpose
User.js	Defines and manages user-related information required for authentication and user management.
Blog.js	Defines and manages blog-related information such as blog content and other required details.

Project Execution

- The Express.js application is initialized.
- The server starts on the configured port.
- The application establishes a connection with MongoDB.
- API routes become available.
- Authentication APIs can be tested.
- Blog APIs can be tested.
- AI APIs can be tested using Gemini AI.
- API responses are verified using Thunder Client.

API Testing and Output Verification

Thunder Client is used to send REST API requests and verify JSON responses during development and testing.

Operation	Expected Purpose
Register	Create a new user account.
Login	Authenticate the user and obtain a JWT token.
Create Blog	Create and store a blog post.
Get / Fetch Blog	Retrieve stored blog information.
Generate Blog	Generate blog content using Gemini AI.
Summarize Blog	Generate a short summary from blog content.

Demo Video

The supplied project documentation provides a Google Drive demo video showing the working of the AI BlogNest API.

Demo Video: https://drive.google.com/file/d/1MhCqIg10Q5YqpqMNdZpT8p_pwEBKDYQM/view?usp=drive

The demonstration covers user registration, login, blog management, AI-powered blog generation, and AI-powered blog summarization.

Future Enhancements

- Development of a user-friendly frontend application.
- AI-based blog title generation.
- AI-based grammar and content improvement.
- AI-generated images for blog posts.
- Advanced blog search and filtering.
- Categories and tags for better blog organization.
- Comments and likes for user interaction.
- User profile management.
- Admin dashboard for managing users and blogs.
- Personalized AI-based blog recommendations.
- Cloud deployment of the backend application.
- Additional AI-powered content management features.

Conclusion

AI BlogNest API provides a modular backend foundation for a blog application by combining secure authentication, MongoDB-based data management, RESTful APIs, and Google Gemini AI capabilities. The project supports blog management together with AI-powered content generation and summarization, while its modular structure supports maintainability and future enhancement.

Final Project Information

Project Name	AI BlogNest API
Team ID	SWTID-2026-1876
Date	29 September 2026
Backend	Node.js + Express.js
Database	MongoDB + Mongoose
AI Integration	Google Gemini AI
Authentication	JWT + bcrypt
API Testing	Thunder Client

Project Reference Summary

This document uses the supplied AI BlogNest API project documentation as the source for project-specific content. The structure follows the clean, white-page, black-text report style shown in the supplied reference screenshot.

Project Scope

- Backend API development for an AI-assisted blog application.
- Secure authentication and protected API operations.
- Blog CRUD functionality.
- Gemini AI integration for generation and summarization.
- MongoDB data storage using Mongoose.
- API verification using Thunder Client.

Source-Supported Information

Where the supplied documentation does not provide a hosted URL, demo login credentials, or a specific repository URL, this document leaves the information as unspecified rather than creating unsupported details.

This keeps the project report aligned with the information actually provided in the project documentation.

End of Project Documentation